

StanThrust Solver Notes

Detailed Geometry, Feed-System, Cooling, and Quasi-1D Performance Formulation

StanThrust Documentation Package

June 17, 2026

1 Purpose

StanThrust is a liquid-engine sizing and analysis tool with a deliberately fast reduced-order solver stack. The software takes a mission request, converts it into a geometry bundle, enforces an architecture-aware feed-pressure state, evaluates a burn-time feed transient, calls a thermochemistry provider, and then evaluates thrust, impulse, structural margins, and export geometry with a quasi-one-dimensional chamber and nozzle model. The software now exposes both a fast preview-oriented flow mode and a refined quasi-one-dimensional mode for explicit solves, while keeping benchmark and regression cases machine-readable and reproducible.

This report is written as an engineering methods note for the *current StanThrust solver*. The goal is to show, in explicit algebraic form, how the program moves from a mission request to geometry, flow quantities, pressures, and performance estimates. The representative plots in this document come from the default packaged case:

$$\text{fuel} = \text{Ethanol}, \quad \text{oxidizer} = \text{Liquid Oxygen}, \quad \left(\frac{O}{F}\right) = 1.40, \quad F_{\text{target}} = 250.0 \text{ N}, \quad t_b = 12.00 \text{ s}.$$

The same reduced-order histories used to generate the report figures are also exposed directly in the Qt desktop application through a dedicated plots tab, so the packaged UI and this document refer to the same transient, axial, and convergence data products.

When reading the report, each section can be interpreted in the same sequence used during preliminary engine sizing:

1. start from the mission targets and packaging constraints;
2. convert those into geometry and propellant requirements;
3. check that the feed system can support the resulting chamber pressure;
4. evaluate how the feed state drifts across the burn;
5. evaluate the nozzle and chamber performance model;
6. compare the resulting scale against benchmark hardware.

2 Pipeline Overview

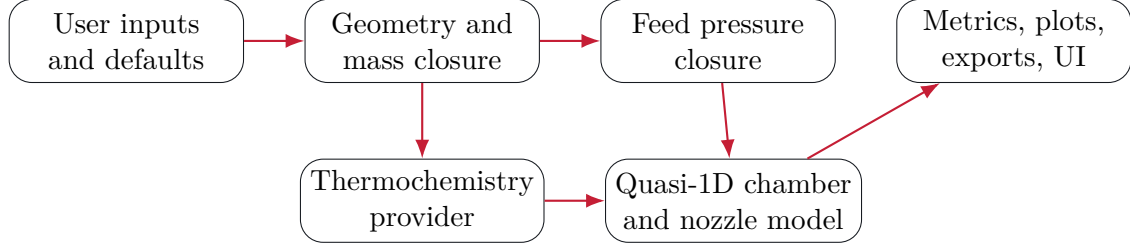


Figure 1: Current StanThrust solver flow.

The software is therefore a staged closure:

1. normalize the input request;
2. generate a first-pass geometry and flow state;
3. enforce feed feasibility for either pump-fed or pressure-fed architecture;
4. obtain (γ, R, T_c) from thermochemistry;
5. compute nozzle performance, mass flow, and predicted thrust.

3 Symbols and Conventions

Table 1: Primary symbols used in the StanThrust formulation.

Symbol	Meaning
F_{target}	user-requested thrust
I_{target}	user-requested total impulse
t_b	burn time
P_c	chamber pressure
A_c, A_t, A_e	chamber, throat, and exit area
d_c, d_t, d_e	chamber, throat, and exit diameter
L_c, L_n	chamber and nozzle length
γ	ratio of specific heats from thermochemistry
R	specific gas constant from thermochemistry
T_c	chamber temperature from thermochemistry
c^*	characteristic velocity
C_F	thrust coefficient
$\dot{m}_p$	total propellant mass flow
$\dot{m}_f, \dot{m}_{ox}$	fuel and oxidizer mass flow
ε	expansion ratio A_e/A_t
t_w	wall thickness

Whenever StanThrust constrains a value between lower and upper bounds, this report writes the operation as

$$\text{clip}(x, x_{\min}, x_{\max}) = \min(x_{\max}, \max(x_{\min}, x)).$$

4 Input Normalization and Scaling

The initial sizing logic does not work directly on raw thrust and impulse. It first converts them into bounded nondimensional scaling factors:

$$S_F = \text{clip} \left(\left(\frac{F_{\text{target}}}{250} \right)^{0.22}, 0.7, 2.4 \right) \quad (1)$$

$$S_I = \text{clip} \left(\left(\frac{I_{\text{target}}}{3000} \right)^{0.16}, 0.75, 2.2 \right) \quad (2)$$

$$S_b = 0.72 + \frac{t_b}{28} \quad (3)$$

$$\beta_{\text{mix}} = \text{clip} \left(\frac{\left(\frac{O}{F} \right)}{1 + \left(\frac{O}{F} \right)}, 0.18, 0.86 \right). \quad (4)$$

Additional multipliers are derived from propellant catalog properties and user-selected architecture:

$$\phi_f = 1 + (0.62 - \rho_f^*) 0.42, \quad \phi_{ox} = 1 + (0.68 - \rho_{ox}^*) 0.36 \quad (5)$$

where ρ^* denotes the catalog density index rather than dimensional density.

The architecture and cooling modifiers are

$$\phi_{\text{blowdown}} = \begin{cases} 1.0, & \text{pump-fed} \\ 1.2, & \text{pressure-fed} \end{cases} \quad \phi_{\text{regen}} = \begin{cases} 0.92, & \text{regen enabled} \\ 1.0, & \text{otherwise} \end{cases} \quad \phi_{\text{film}} = \begin{cases} 1.04, & \text{film cooling enabled} \\ 1.0, & \text{otherwise} \end{cases} \quad (6)$$

and the propellant thermal modifier is

$$\phi_T = 1 + 0.12 \tau_{ox} - 0.08 \chi_f \quad (7)$$

with τ_{ox} the oxidizer thermal-severity index and χ_f the fuel cooling-affinity index from the internal propellant catalog.

5 Geometry Closure

5.1 Tank, Chamber, and Nozzle Lengths

The first-pass axial layout is generated from multiplicative closures. The fuel and oxidizer tank lengths are

$$L_{f,\text{tank}} = d_{\text{tank}} \Pi_{\text{tank}} S_b S_I \phi_f (1.04 - 0.22 \beta_{\text{mix}}) \phi_{\text{blowdown}} \quad (8)$$

$$L_{ox,\text{tank}} = d_{\text{tank}} \Pi_{\text{tank}} S_b S_I \phi_{ox} (0.88 + 0.36 \beta_{\text{mix}}) \phi_{\text{blowdown}} \quad (9)$$

where Π_{tank} is the packaging multiplier associated with the selected packaging bias.

The chamber length is

$$L_c = d_c \Pi_{\text{chamber}} S_F \phi_T \phi_{\text{regen}} \quad (10)$$

with Π_{chamber} the chamber-length packaging multiplier.

StanThrust also treats the user-specified target diameter as a hard outer-envelope limit. If D_{req} is the uncapped required outside diameter of a tank, chamber, or nozzle package and D_{lim} is the target diameter, the exported geometry uses

$$D_{\text{used}} = \min(D_{\text{req}}, D_{\text{lim}}), \quad (11)$$

while still reporting D_{req} separately in the diagnostics and export data. This makes envelope compliance explicit instead of hiding it in rounded UI output.

The nozzle throat diameter is

$$d_t = \text{clip}(d_c (0.38 + 0.03 S_F), 10.0, 0.72 d_e) \quad (12)$$

and the converging and diverging lengths are

$$L_{\text{conv}} = \text{clip}(d_c (0.42 + 0.07 S_F), 10.0, 1.55 d_c) \quad (13)$$

$$L_{\text{div}} = \text{clip}(d_e (0.58 + 0.09 S_F) \phi_{\text{film}}, 18.0, 1.8 d_e) \quad (14)$$

$$L_n = L_{\text{conv}} + L_{\text{div}}. \quad (15)$$

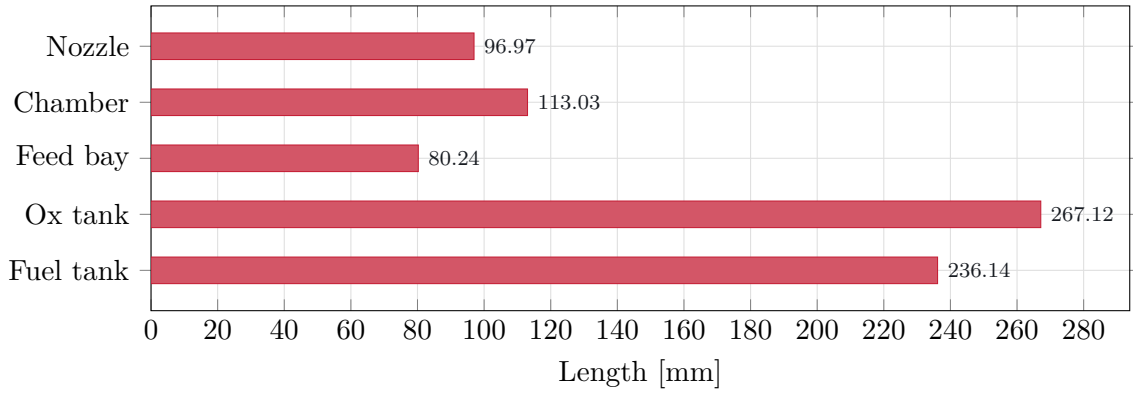


Figure 2: Axial length breakdown of the default layout.

The resulting geometric expansion ratio is

$$\varepsilon = \left(\frac{d_e}{d_t} \right)^2 \quad (16)$$

and the half-angles inferred from the linear envelope are

$$\theta_{\text{conv}} = \text{clip} \left[\tan^{-1} \left(\frac{d_c - d_t}{2L_{\text{conv}}} \right) \frac{180}{\pi}, 12, 38 \right] \quad (17)$$

$$\theta_{\text{div}} = \text{clip} \left[\tan^{-1} \left(\frac{d_e - d_t}{2L_{\text{div}}} \right) \frac{180}{\pi}, 6, 22 \right]. \quad (18)$$

5.2 Nozzle Contour Construction

StanThrust renders the actual nozzle contour as a sampled axial-radius polyline rather than a triangular sketch. The current method is

MOC-informed bell contour.

Let

$$r_c = \frac{d_c}{2}, \quad r_t = \frac{d_t}{2}, \quad r_e = \frac{d_e}{2}.$$

The converging half-angle is reconstructed from the solved converging length:

$$\theta_c = \tan^{-1} \left(\frac{r_c - r_t}{L_{\text{conv}}} \right). \quad (19)$$

StanThrust then assigns throat blend radii

$$R_{\text{up}} = \max(1.25 r_t, r_t + 2), \quad R_{\text{down}} = \max(0.382 r_t, 2) \quad (20)$$

with all lengths in millimeters.

The upstream tangent point on the converging throat blend is

$$x_{\text{up}} = L_{\text{conv}} - R_{\text{up}} \sin \theta_c, \quad r_{\text{up}} = r_t + R_{\text{up}} (1 - \cos \theta_c) \quad (21)$$

and the straight converging segment begins at

$$x_s = x_{\text{up}} - \frac{r_c - r_{\text{up}}}{\tan \theta_c}. \quad (22)$$

The converging line segment is therefore

$$r_{\text{line}}(x) = r_c - \tan \theta_c (x - x_s), \quad x_s \leq x \leq x_{\text{up}} \quad (23)$$

and the throat-entry arc is

$$x_{\text{up-arc}}(\phi) = L_{\text{conv}} - R_{\text{up}} \sin \phi, \quad r_{\text{up-arc}}(\phi) = r_t + R_{\text{up}} (1 - \cos \phi), \quad 0 \leq \phi \leq \theta_c. \quad (24)$$

The diverging contour is treated as a bell nozzle referenced to a 15° conical nozzle of equal throat and exit radii:

$$L_{15} = \frac{r_e - r_t}{\tan 15^\circ} \quad (25)$$

$$f_{\text{bell}} = \text{clip} \left(\frac{L_{\text{div}}}{L_{15}}, 0.55, 1.10 \right). \quad (26)$$

For the default case,

$$L_{15} = 125.2 \text{ mm}, \quad f_{\text{bell}} = 0.550.$$

The bell entrance and exit angles are

$$\theta_n = \text{clip}(29 - 0.35 \max(\varepsilon - 8, 0) + 18(0.82 - f_{\text{bell}}), 24, 33) \quad (27)$$

$$\theta_e = \text{clip}(14.5 - 1.25 \ln \varepsilon, 6, 14). \quad (28)$$

For the packaged sample,

$$\theta_n = 34.00^\circ, \quad \theta_e = 9.37^\circ, \quad R_{\text{up}} = 17.4 \text{ mm}, \quad R_{\text{down}} = 5.3 \text{ mm}.$$

The initial diverging throat arc is

$$x_{\text{down-arc}}(\phi) = R_{\text{down}} \sin \phi, \quad r_{\text{down-arc}}(\phi) = r_t + R_{\text{down}} (1 - \cos \phi), \quad 0 \leq \phi \leq \theta_n. \quad (29)$$

Its terminal point is denoted by

$$P_0 = (x_0, r_0), \quad P_2 = (L_{\text{div}}, r_e).$$

StanThrust then constructs a quadratic bell from P_0 to P_2 by intersecting the entrance and exit tangent lines. With

$$m_n = \tan \theta_n, \quad m_e = \tan \theta_e,$$

the control point $P_1 = (x_1, r_1)$ is

$$x_1 = \frac{r_e - m_e L_{\text{div}} - r_0 + m_n x_0}{m_n - m_e}, \quad r_1 = r_0 + m_n(x_1 - x_0). \quad (30)$$

The bell profile is then sampled from the quadratic Bézier curve

$$\mathbf{P}(t) = (1-t)^2 \mathbf{P}_0 + 2(1-t)t \mathbf{P}_1 + t^2 \mathbf{P}_2, \quad 0 \leq t \leq 1 \quad (31)$$

and shifted downstream by the converging length so that the throat remains at $x = L_{\text{conv}}$ in the full nozzle coordinate system.

For the report case,

$$L_c = 113.0 \text{ mm}, \quad L_n = 97.0 \text{ mm}, \quad d_t = 27.9 \text{ mm}, \quad d_e = 95.0 \text{ mm}, \quad \varepsilon = 11.611.$$

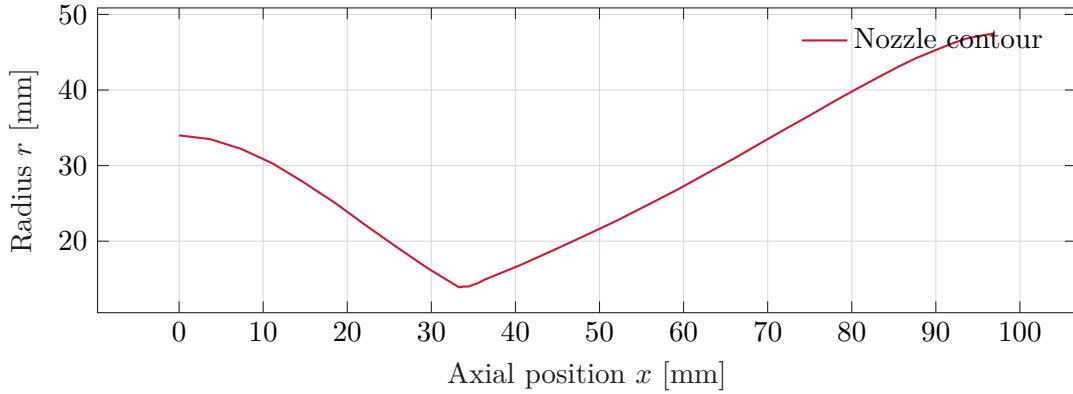


Figure 3: Sampled MOC-informed bell contour generated from the current geometry closure.

5.3 Overall Stack and Surface Proxies

The total engine stack is

$$L_{\text{stack}} = L_{f,\text{tank}} + L_{ox,\text{tank}} + L_c + L_n + L_{\text{feed}} + \Delta L_{\text{aft}} \quad (32)$$

where

$$L_{\text{feed}} = \begin{cases} 1.18 d_c, & \text{pump-fed} \\ 1.02 d_{\text{tank}}, & \text{pressure-fed} \end{cases} \quad \Delta L_{\text{aft}} = \begin{cases} 18, & \text{pump-fed} \\ 32, & \text{pressure-fed} \end{cases} \quad (33)$$

in millimeters.

The chamber and nozzle shell areas used in the mass and complexity terms are

$$A_{s,c} = 2\pi \left(\frac{d_c}{2} \right) L_c \quad (34)$$

$$A_{s,n} = 0.82 \pi d_e L_n. \quad (35)$$

5.4 Renderable Geometry Fields

The desktop 3D views are rendered from the same solved scalar fields and sampled nozzle contour used by the measurement table and exports. The drawing layer may choose colors, perspective, wireframe density, and label positions, but the visible engine dimensions are not hard-coded in the renderer. For example, tank cylinders use solved outer diameters and required lengths, pump models use solved casing, impeller, eye, hub, blade-count, and motor-envelope values, and the impinging injector pattern uses the solved element count, orifice diameter, pair spacing, ring diameter, and convergence height.

Table 2: Selected solved fields used directly by the live 3D renderer for the default pump-fed impinging-injector case.

Component	Dimension	Value [mm]
Fuel feed tube	Diameter	4.40
Ox feed tube	Diameter	4.30
Fuel pump casing	Diameter	72.70
Ox pump casing	Diameter	76.70
Fuel impeller	Diameter	67.50
Ox impeller	Diameter	71.10
Motor envelope	Length	77.10
Injector face	Diameter	70.00
Injector recess	Diameter	59.70
Injector element ring	Diameter	50.40

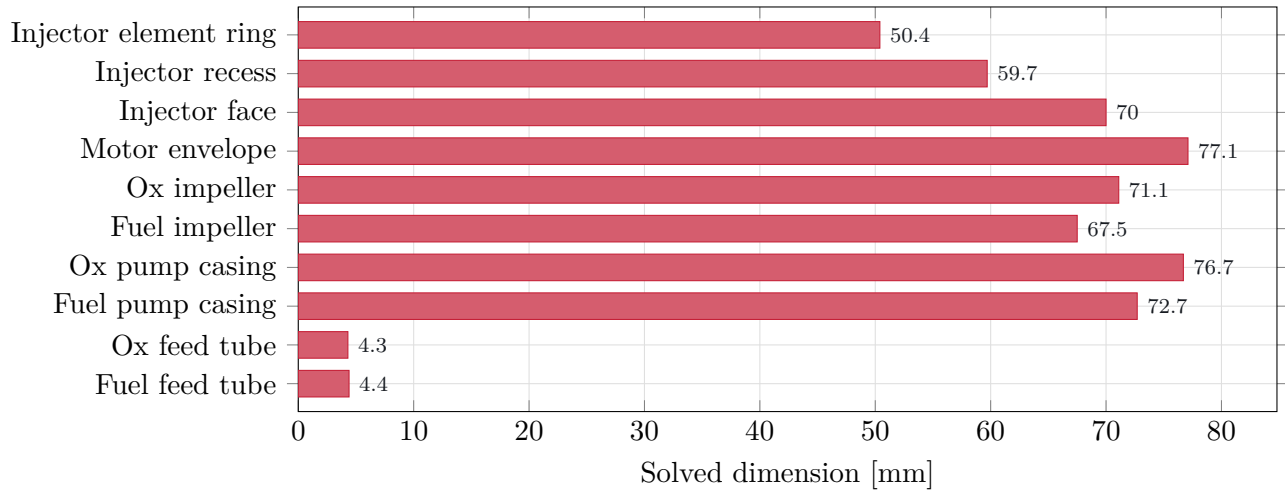


Figure 4: Representative solved dimensions that drive the in-app 3D render. The plotted values are regenerated from the current solver implementation.

6 Performance and Propellant Closure

6.1 Delivered Thrust and Burn-Time Consistency

StanThrust does not assume the requested thrust is exactly delivered. Instead it constructs a thrust-delivery factor

$$\eta_F = \text{clip}\left(0.86 + 0.03\mathbb{K}_{\text{pump}} - 0.02\mathbb{K}_{\text{pressure-fed}} + 0.04\mathbb{K}_{\text{regen}} + 0.02\mathbb{K}_{\text{film}} + 0.05\chi_f - 0.03\tau_{ox}, 0.75, 1.05\right) \quad (36)$$

so that

$$F_{\text{calc}} = \eta_F F_{\text{target}}. \quad (37)$$

The burn time is then recomputed from the impulse request,

$$t_{b,\text{calc}} = \max\left(0.5, \frac{I_{\text{target}}}{\max(1, F_{\text{calc}})}\right) \quad (38)$$

and the internally consistent impulse becomes

$$I_{\text{calc}} = F_{\text{calc}} t_{b,\text{calc}}. \quad (39)$$

6.2 Effective Exhaust Velocity and Mass Flow

Before the quasi-1D solver is run, the geometry model estimates an effective exhaust velocity with

$$v_{e,\text{eff}} = \text{clip}(1720 + 3.8 M_T + 85\mathbb{K}_{\text{pump}} - 20\mathbb{K}_{\text{pressure-fed}} + 40\mathbb{K}_{\text{regen}}, 1400, 2450) \quad (40)$$

where M_T is the thermal-margin index described later.

The propellant mass flow and total consumed mass are then

$$\dot{m}_p = \frac{F_{\text{calc}}}{v_{e,\text{eff}}} \quad (41)$$

$$m_p = \dot{m}_p t_{b,\text{calc}}. \quad (42)$$

Given mixture ratio $\left(\frac{O}{F}\right)$, the fuel and oxidizer split is

$$m_f = \frac{m_p}{1 + \left(\frac{O}{F}\right)}, \quad m_{ox} = m_p - m_f \quad (43)$$

$$\dot{m}_f = \frac{\dot{m}_p}{1 + \left(\frac{O}{F}\right)}, \quad \dot{m}_{ox} = \dot{m}_p - \dot{m}_f. \quad (44)$$

6.3 Tank Volume and Volume-Flow Closure

Using catalog dimensional densities ρ_f and ρ_{ox} , StanThrust computes ullage-modified tank volumes:

$$\phi_u = \begin{cases} 1.08, & \text{pump-fed} \\ 1.16, & \text{pressure-fed} \end{cases} \quad (45)$$

$$V_f = \frac{m_f}{\rho_f} \phi_u, \quad V_{ox} = \frac{m_{ox}}{\rho_{ox}} \phi_u. \quad (46)$$

The dimensional volume flow rates are

$$\dot{V}_f = \frac{\dot{m}_f}{\rho_f}, \quad \dot{V}_{ox} = \frac{\dot{m}_{ox}}{\rho_{ox}}. \quad (47)$$

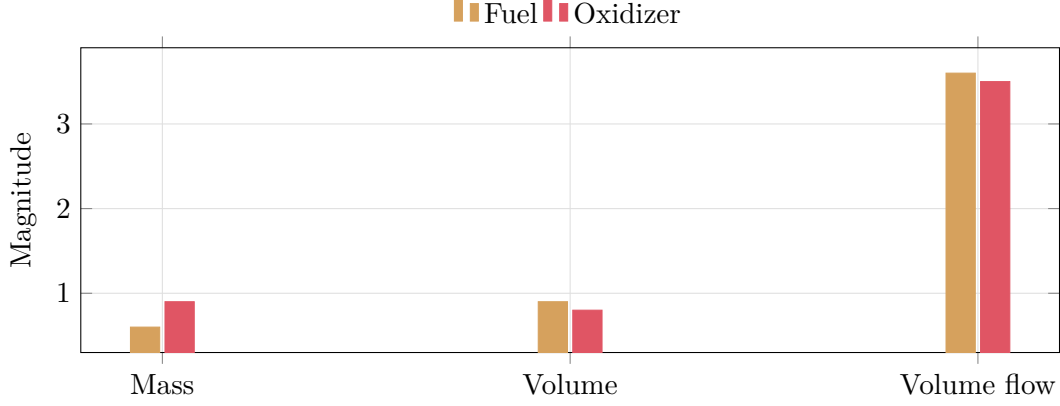


Figure 5: Fuel and oxidizer inventory breakdown for the default case. The three groups correspond to stored mass, stored volume, and volume flow rate.

7 Structural and Tank Sizing Relations

7.1 Wall Thickness

Tank and shell wall thicknesses use thin-wall hoop-stress logic:

$$t_w = \frac{Pr \text{ FOS}}{\sigma_{\text{allow}}} \quad (48)$$

with P in pascals, r in meters, and σ_{allow} the allowable stress for the selected material. In code, the result is converted to millimeters and constrained to

$$t_{w,\text{used}} = \text{clip}(t_w, 1 \text{ mm}, 16 \text{ mm}). \quad (49)$$

7.2 Tank Internal Diameter and Required Length

If d_{tank} is the user-facing outer diameter, then the internal diameter of each tank is

$$d_{f,\text{ID}} = \max(16, d_{\text{tank}} - 2t_{w,f}), \quad d_{ox,\text{ID}} = \max(16, d_{\text{tank}} - 2t_{w,ox}) \quad (50)$$

in millimeters.

The cross-sectional areas are

$$A_f = \pi \left(\frac{d_{f,\text{ID}}}{2000} \right)^2, \quad A_{ox} = \pi \left(\frac{d_{ox,\text{ID}}}{2000} \right)^2 \quad (51)$$

with diameters converted to meters, and the corresponding required lengths are

$$L_{f,\text{req}} = \frac{V_f}{A_f} \times 1000, \quad L_{ox,\text{req}} = \frac{V_{ox}}{A_{ox}} \times 1000 \quad (52)$$

in millimeters.

8 Architecture Metrics and Proxy Indices

StanThrust also maintains bounded engineering indices used in the UI and optimization summaries.

The complexity index is

$$I_{\text{complex}} = \text{clip}\left(34 + 12\mathbb{K}_{\text{pump}} + 8\mathbb{K}_{\text{pressure-fed}} + 14\mathbb{K}_{\text{regen}} + 9\mathbb{K}_{\text{film}} + 12(h_f + h_{ox}), 20, 95\right) \quad (53)$$

where h_f and h_{ox} are propellant handling-complexity indices.

The dry-mass index is

$$I_{\text{dry}} = \text{clip}\left(18 + \frac{A_{s,c}}{6200} + \frac{A_{s,n}}{9000} + \frac{L_{\text{stack}}}{42} + 0.45 I_{\text{complex}}, 12, 98\right) \quad (54)$$

and the packaging-efficiency index is

$$I_{\text{pack}} = \text{clip}\left(92 - 2.2 \frac{L_{\text{stack}}}{\max(d_{\text{tank}}, d_e)} - 0.14 I_{\text{complex}} + 4\mathbb{K}_{\text{compact}} + 2\mathbb{K}_{\text{pump}} - 1.5\mathbb{K}_{\text{pressure-fed}}, 5, 96\right). \quad (55)$$

With chamber loading

$$\lambda_c = \frac{L_c}{d_c} \quad (56)$$

the thermal-margin index is

$$M_T = \text{clip}\left(54 + 18\chi_f - 12\tau_{ox} + 16\mathbb{K}_{\text{regen}} + 8\mathbb{K}_{\text{film}} - 4.5\lambda_c - 3\mathbb{K}_{\text{pressure-fed}}, 8, 94\right). \quad (57)$$

9 Feed Pressure Closure

The feed-pressure solver is executed before the chamber/nozzle model, but it is no longer a single steady algebraic closure. StanThrust now advances a reduced-order feed state over burn time and stores the resulting pressure, margin, and pump-state histories.

9.1 Common Relations

At every time step, the nominal total propellant flow is split by mixture ratio:

$$\dot{m}_p = \frac{T}{g_0 I_{sp, \text{proxy}}}, \quad \dot{m}_{ox} = \frac{(O/F)}{1 + O/F} \dot{m}_p, \quad \dot{m}_f = \dot{m}_p - \dot{m}_{ox}. \quad (58)$$

Line losses are computed branch by branch with a deterministic Darcy-Weisbach closure:

$$\Delta P_\ell = \left(f \frac{L}{D} + K_{\text{minor}}\right) \frac{\rho v^2}{2}, \quad v = \frac{\dot{m}}{\rho A}, \quad A = \frac{\pi D^2}{4}. \quad (59)$$

The Reynolds number and friction factor are evaluated as

$$\text{Re} = \frac{\rho v D}{\mu}, \quad f = \begin{cases} 64/\text{Re}, & \text{Re} < 2300, \\ 0.3164 \text{Re}^{-1/4}, & \text{Re} \geq 2300. \end{cases} \quad (60)$$

Injector pressure drop remains proportional to chamber pressure:

$$\Delta P_{\text{inj}} = \chi_{\text{inj}} P_c, \quad \chi_{\text{inj}} = \begin{cases} 0.18, & \text{Impinging}, \\ 0.14, & \text{Pintle}. \end{cases} \quad (61)$$

The upstream pressure that the feed system must supply is then

$$P_{\text{req}} = P_c + \Delta P_{\text{inj}} + \max(\Delta P_{\ell,f}, \Delta P_{\ell,ox}). \quad (62)$$

Because the chamber pressure and mass flow are coupled, StanThrust iterates each time step with

$$\phi_k = \text{clip}\left(\frac{P_{c,k}}{P_{c,0}}, 0.38, 1.12\right), \quad \dot{m}_{p,k} = \phi_k \dot{m}_{p,0}, \quad \Delta P_{\ell,k} = \phi_k^2 \Delta P_{\ell,0}. \quad (63)$$

This keeps the transient stable while still allowing line losses and chamber pressure to drift together over the burn.

9.2 Pressure-Fed Blowdown Branch

For a pressure-fed case, each tank is treated as a gas-backed liquid volume with an initial liquid fill fraction β . If $\zeta = t/t_b$ is the consumed burn fraction, the gas-volume ratio becomes

$$\frac{V_g(\zeta)}{V_{g,0}} = 1 + \zeta \frac{\beta}{1 - \beta}, \quad (64)$$

and the reduced-order polytropic blowdown law is

$$P_t(\zeta) = P_a + \frac{P_{t,0} - P_a}{(V_g(\zeta)/V_{g,0})^n}, \quad (65)$$

where n is the pressurant polytropic index and P_a is ambient pressure.

At each time step, the available feed pressure after branch losses is

$$P_{\text{avail},k} = \min(P_{t,f,k} - \Delta P_{\ell,f,k}, P_{t,ox,k} - \Delta P_{\ell,ox,k}), \quad (66)$$

and the chamber can sustain at most

$$P_{c,k} = \text{clip}\left(\min\left(P_{c,0}, \frac{P_{\text{avail},k}}{1 + \chi_{\text{inj}}}\right), P_a + 35, P_{c,0}\right). \quad (67)$$

Fuel-side and oxidizer-side feed margins are reported explicitly:

$$\Delta P_{m,f,k} = P_{t,f,k} - P_{\text{req},k}, \quad \Delta P_{m,ox,k} = P_{t,ox,k} - P_{\text{req},k}. \quad (68)$$

These are the quantities plotted and exported, because they directly show whether the tanks remain above the required delivery pressure over the whole burn.

9.3 Pump-Fed Controlled Branch

For a pump-fed case, StanThrust keeps low-pressure inlet tanks and models only the pump-side state history needed to maintain the chamber. The inlet tank pressures are allowed to decay mildly:

$$P_{t,k} = P_a + (P_{t,0} - P_a) (1 - \delta \zeta^{1.15}), \quad (69)$$

where δ is a small prescribed decay fraction.

The required pump discharge pressure is

$$P_{\text{pump,req},k} = P_c + \Delta P_{\text{inj},k} + \max(\Delta P_{\ell,f,k}, \Delta P_{\ell,ox,k}), \quad (70)$$

and the controller applies a small reserve:

$$P_{\text{pump},k} = \min(P_{\text{pump},\text{cap}}, P_{\text{pump},\text{req},k} + \Delta P_{\text{ctrl},k}), \quad (71)$$

with

$$\Delta P_{\text{ctrl},k} = \max(18, [0.025 + 0.015\zeta] P_{\text{pump},\text{req},k}). \quad (72)$$

The pump differential head is then

$$\Delta P_{\text{pump},k} = P_{\text{pump},k} - \frac{1}{2} (P_{t,f,k} + P_{t,ox,k}), \quad (73)$$

and the corresponding nondimensional pump-speed proxy is

$$N_k^* = \text{clip}\left(\frac{P_{\text{pump},k}}{P_{\text{pump},0}}, 0.82, 1.18\right). \quad (74)$$

This is still a reduced-order model, but it produces a usable burn history for pump differential pressure and pump-control effort instead of a single steady value.

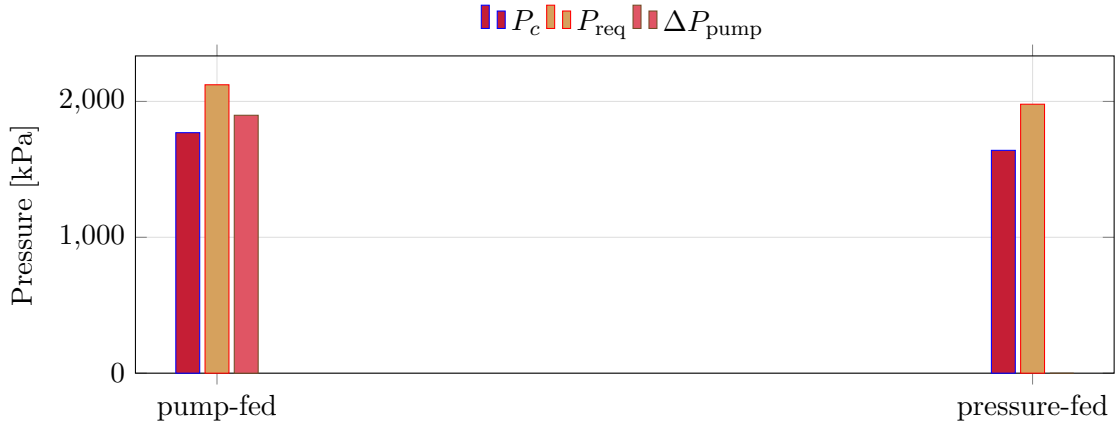


Figure 6: Pressure closure comparison for the same mission request under pump-fed and pressure-fed architectures.

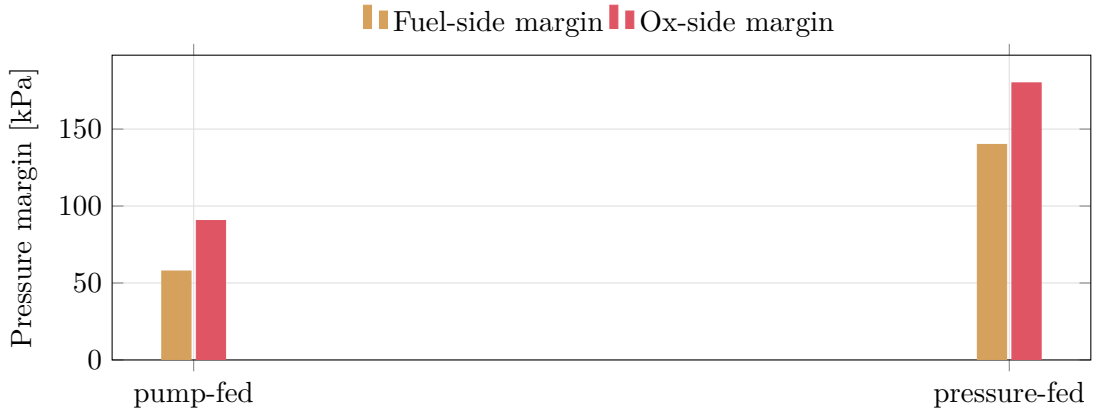


Figure 7: Feed pressure margins retained by the explicit architecture closure.

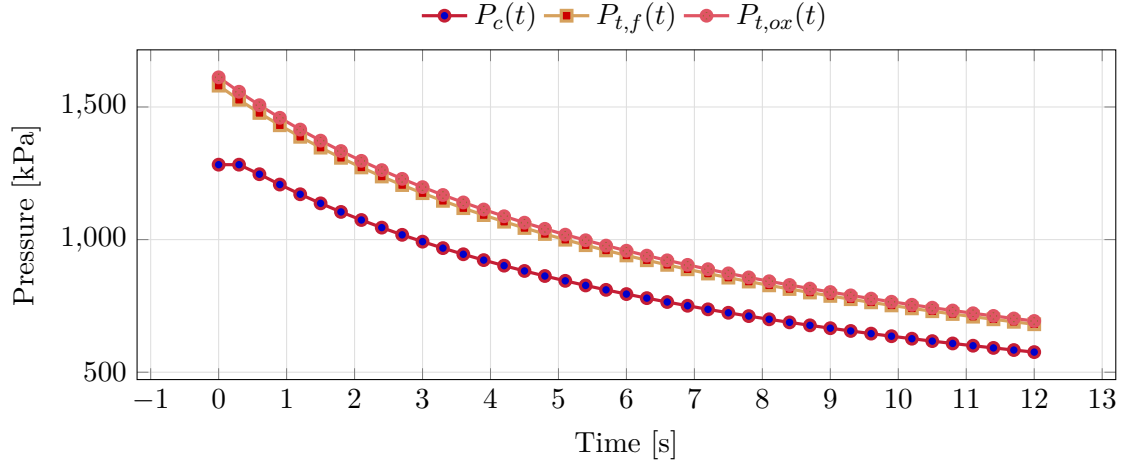


Figure 8: Pressure-fed blowdown history for the default sample case. Tank pressures decay continuously, and the chamber pressure tails off only when the available feed pressure can no longer hold the nominal plateau.

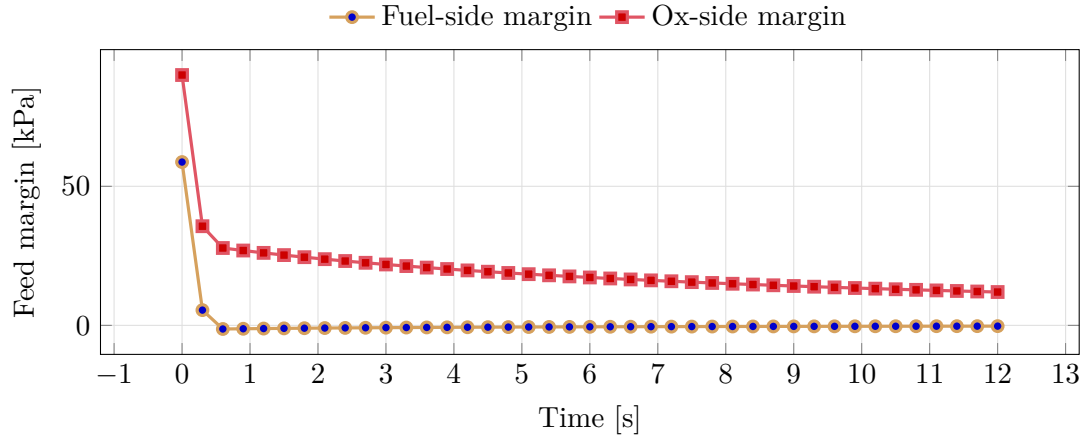


Figure 9: Pressure-fed margin history. The minimum of these two curves is the exported burn-critical feed margin.

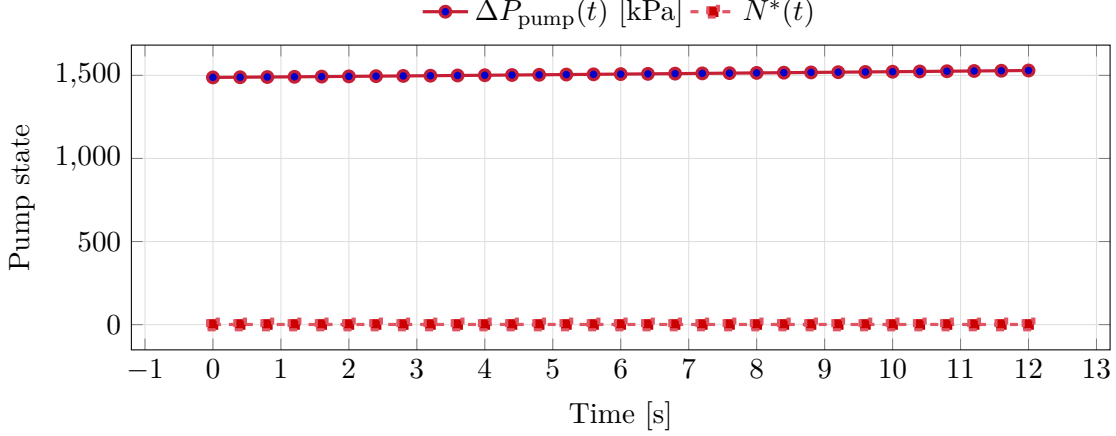


Figure 10: Pump-fed transient history. The chamber pressure remains close to its nominal value while the pump differential pressure and pump-speed proxy drift over the burn.

10 Pump, Injector, and Cooling Submodels

10.1 Pump Geometry and Drive Proxies

If pump-fed mode is active, pump efficiency is estimated as

$$\eta_{\text{pump}} = \text{clip}(0.52 + 0.05\chi_f - 0.02\tau_{ox}, 0.46, 0.78). \quad (75)$$

The impeller diameters are

$$d_{\text{imp},f} = \text{clip}\left(36 + 4.6\sqrt{60000 \dot{V}_f} + 0.012 \Delta P_{\text{pump}}, 28, 0.92 d_{\text{tank}}\right) \quad (76)$$

$$d_{\text{imp},ox} = \text{clip}\left(36 + 4.6\sqrt{60000 \dot{V}_{ox}} + 0.014 \Delta P_{\text{pump}}, 28, 0.92 d_{\text{tank}}\right) \quad (77)$$

with geometry relations

$$w_{\text{imp}} = 0.18 d_{\text{imp}}, \quad d_{\text{hub}} = 0.42 d_{\text{imp}}, \quad d_{\text{eye}} = 0.48 d_{\text{imp}} \quad (78)$$

subject to the same bounds used by the present solver.

Blade counts are piecewise:

$$N_{b,f} = \begin{cases} 5, & \Delta P_{\text{pump}} < 1200 \\ 6, & \Delta P_{\text{pump}} \geq 1200 \end{cases} \quad N_{b,ox} = \begin{cases} 6, & \Delta P_{\text{pump}} < 1600 \\ 7, & \Delta P_{\text{pump}} \geq 1600 \end{cases} \quad (79)$$

and the motor power is

$$P_{\text{motor}} = \text{clip}\left(\frac{\Delta P_{\text{pump}} 1000 (\dot{V}_f + \dot{V}_{ox})}{1000 \max(0.35, \eta_{\text{pump}})}, 0.25, 120\right) \quad [\text{kW}] \quad (80)$$

with speed and torque

$$N_{\text{motor}} = \text{clip}(6000 + 1800S_F, 3000, 18000) \quad (81)$$

$$\tau_{\text{motor}} = 9550 \frac{P_{\text{motor}}}{\max(1000, N_{\text{motor}})}. \quad (82)$$

10.2 Injector Geometry

The injector face diameter and thickness are

$$d_{\text{inj}} = \begin{cases} 1.09 d_c, & \text{film cooling enabled} \\ 1.03 d_c, & \text{otherwise} \end{cases} \quad (83)$$

$$t_{\text{inj}} = \text{clip}(0.075 d_c, 3.5, 16.0). \quad (84)$$

For a pintle injector, the sizing proxies are

$$d_{\text{tip}} = \text{clip}(0.24 d_c, 7.5, 26.0), \quad d_{\text{stem}} = \text{clip}(0.62 d_{\text{tip}}, 5.0, 18.0) \quad (85)$$

$$g_{\text{annulus}} = \text{clip}(0.95 + 2.4 \dot{m}_p, 0.8, 4.2), \quad L_{\text{proj}} = \text{clip}(0.55 d_c, 12.0, 62.0). \quad (86)$$

For an impinging injector, the orifice diameter, angle, and pair spacing are

$$d_{\text{orifice}} = \text{clip}\left(1.2 + 1.6\sqrt{\dot{m}_p}, 1.0, 5.8\right) \quad (87)$$

$$\theta_{\text{imp}} = \text{clip}(28 + 16\beta_{\text{mix}}, 22, 56) \quad (88)$$

$$s_{\text{pair}} = \text{clip}(0.085 d_c, 3.0, 12.0) \quad (89)$$

with the present implementation using two element pairs.

10.3 Regenerative Cooling Proxy

If regenerative cooling is enabled, the jacket thickness is

$$t_{\text{jacket}} = \text{clip}(0.085 d_c, 4.0, 14.0). \quad (90)$$

From this, the internal geometry is

$$h_{\text{ch}} = \text{clip}(0.62 t_{\text{jacket}}, 1.5, 8.0), \quad h_{\text{rib}} = \text{clip}(0.42 t_{\text{jacket}}, 0.8, 5.0) \quad (91)$$

$$t_{\text{inner}} = \text{clip}(0.18 t_{\text{jacket}}, 0.5, 2.0), \quad t_{\text{outer}} = \text{clip}(0.15 t_{\text{jacket}}, 0.4, 1.5) \quad (92)$$

$$p_{\text{ch}} = \text{clip}(0.055 d_e, 3.0, 9.5), \quad w_{\text{ch}} = \text{clip}(p_{\text{ch}} - 0.22 t_{\text{jacket}}, 1.2, p_{\text{ch}} - 0.5) \quad (93)$$

$$t_{\text{rib}} = \text{clip}(p_{\text{ch}} - w_{\text{ch}}, 0.8, 3.5). \quad (94)$$

The channel count is

$$N_{\text{ch}} = \max\left(18, \text{round}\left(\frac{\pi d_e}{p_{\text{ch}}}\right)\right) \quad (95)$$

and the coolant mass flow is

$$\dot{m}_{\text{cool}} = \text{clip}(\dot{m}_f(0.22 + 0.12\tau_{ox}), 0.01, 0.85\dot{m}_f). \quad (96)$$

The effective coolant density is approximated as

$$\rho_{\text{cool}} = \max(120, 0.82 \rho_f) \quad (97)$$

and the available flow area is

$$A_{\text{cool}} = \max\left(10^{-6}, N_{\text{ch}} w_{\text{ch}} h_{\text{ch}} \times 10^{-6}\right). \quad (98)$$

Coolant velocity and hydraulic diameter follow:

$$u_{\text{cool}} = \frac{\dot{m}_{\text{cool}}}{\rho_{\text{cool}} A_{\text{cool}}} \quad (99)$$

$$D_h = \max\left(0.5, \frac{2w_{\text{ch}}h_{\text{ch}}}{w_{\text{ch}} + h_{\text{ch}}}\right). \quad (100)$$

The cooling pressure-drop estimate is

$$\Delta P_{\text{regen}} = \text{clip}\left(\frac{0.018 u_{\text{cool}}^2 L_n}{1000 \max(1, D_h)}, 20, 900\right) \quad [\text{kPa}] \quad (101)$$

and the total radial build is

$$t_{\text{regen,rad}} = t_{\text{inner}} + h_{\text{ch}} + t_{\text{outer}}. \quad (102)$$

The heat-flux and temperature-rise proxies are

$$\dot{Q}_{\text{regen}} = \text{clip}(\dot{m}_p (0.35 + 0.22\chi_f + 0.48\tau_{ox}) 410, 18, 720) \quad [\text{kW}] \quad (103)$$

$$\Delta T_{\text{cool}} = \text{clip}\left(\frac{\dot{Q}_{\text{regen}}}{\dot{m}_{\text{cool}} c_{p,\text{cool}}}, 4, 320\right), \quad c_{p,\text{cool}} = 3.75 \text{ kJ kg}^{-1} \text{ K}^{-1}. \quad (104)$$

Thermal margins are reported by section:

$$M_{\text{feed}} = \text{clip}(M_T + 8, 8, 98) \quad (105)$$

$$M_{\text{pump}} = \text{clip}(M_T + \delta_{\text{arch}} - 0.002 \Delta P_{\text{regen}}, 8, 98) \quad (106)$$

$$M_{\text{inj}} = \text{clip}(M_T - 3.5 - 2.2\mathcal{K}_{\text{film}}, 8, 98) \quad (107)$$

$$M_{\text{chamber}} = \text{clip}(M_T - 7 + 0.42 t_{\text{regen,rad}}, 8, 98) \quad (108)$$

$$M_{\text{throat}} = \text{clip}(M_T - 14 + 0.85 h_{\text{ch}} - 0.01 \Delta P_{\text{regen}}, 8, 98) \quad (109)$$

$$M_{\text{exit}} = \text{clip}\left(M_T - 4 + \frac{L_{\text{div}}}{78}, 8, 98\right) \quad (110)$$

where $\delta_{\text{arch}} = 4.5$ for pump-fed and 1.0 for pressure-fed mode.

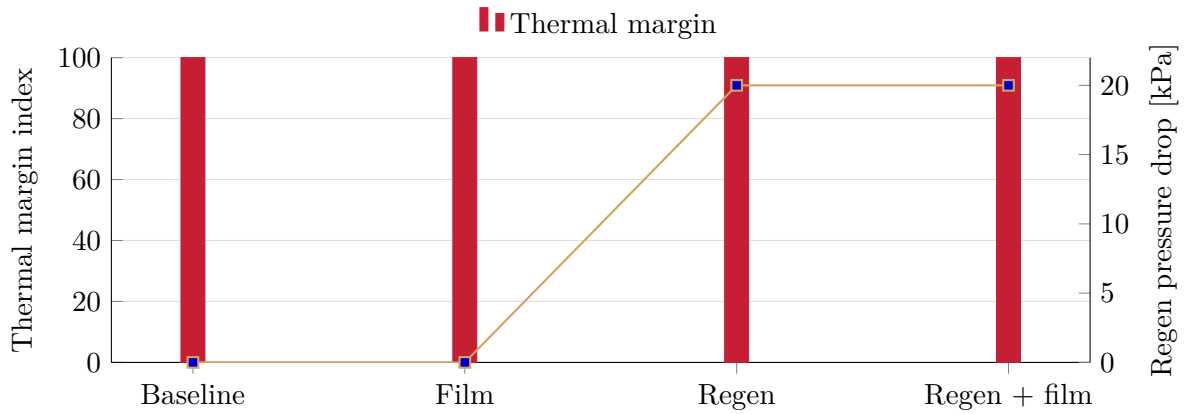


Figure 11: Cooling-mode sweep showing how thermal margin and regenerative-cooling pressure drop change when film and regen options are toggled.

10.4 Film Cooling Proxy

If film cooling is enabled, the film mass flow is

$$\dot{m}_{\text{film}} = \text{clip}(\dot{m}_p(0.045 + 0.025\tau_{ox}), 0.002, 0.18\dot{m}_p) \quad (111)$$

and the slot geometry is

$$h_{\text{slot}} = \text{clip}(0.018 d_{\text{inj}}, 0.25, 1.8) \quad (112)$$

$$w_{\text{slot}} = \text{clip}(0.32\pi d_{\text{inj}}, 12.0, 190.0). \quad (113)$$

The injection angle and coverage fraction are

$$\theta_{\text{film}} = \text{clip}(24 + 16\beta_{\text{mix}}, 18, 42) \quad (114)$$

$$\Gamma_{\text{film}} = \text{clip}(0.42 + 0.12\chi_f, 0.35, 0.90). \quad (115)$$

The effective film flow area and injection velocity are then

$$A_{\text{film}} = \max\left(10^{-6}, \frac{h_{\text{slot}}}{1000} \frac{w_{\text{slot}}}{1000} \Gamma_{\text{film}}\right) \quad (116)$$

$$u_{\text{film}} = \text{clip}\left(\frac{\dot{m}_{\text{film}}}{\rho_f A_{\text{film}}}, 4, 120\right) \quad (117)$$

and the slot count estimate is

$$N_{\text{slot}} = \max(8, \text{round}(0.55 d_{\text{inj}})). \quad (118)$$

11 Thermochemistry

StanThrust requests thermochemistry from the bundled Cantera equilibrium mechanism

```
rocket_mech_equilibrium.yaml
```

when available. The solver consumes the provider outputs

$$\gamma = 1.2500, \quad R = 420.0 \text{ J kg}^{-1} \text{ K}^{-1}, \quad T_c = 3200.0 \text{ K}.$$

These are not reconstructed inside the geometry model; they are taken from the thermochemistry provider and inserted directly into the quasi-one-dimensional performance model. If Cantera is unavailable, the application uses a bounded approximate fallback so the UI remains operational.

12 Quasi-One-Dimensional Chamber and Nozzle Solver

12.1 Fast and Refined Flow Modes

StanThrust now exposes two internal flow modes:

1. **Fast preview mode**, which preserves the original constant-coefficient quasi-one-dimensional closure and is intended for quick design iteration.
2. **Refined mode**, which uses the solved throat diameter directly, applies a contour-aware bell-nozzle loss model, and subtracts an ambient-pressure correction from the vacuum thrust coefficient.

For the report case, the detailed derivation below uses

Refined quasi-1D solve.

The refined mode modifies the delivered thrust coefficient according to

$$C_{F,\text{amb}} = C_{F,\text{vac}} - \left(\frac{P_a}{P_c} \right) \varepsilon \quad (119)$$

and then applies a separation-efficiency term when the exit pressure becomes substantially lower than ambient:

$$\eta_{\text{sep}} = \begin{cases} \text{clip}\left(0.82 + 0.24 \frac{P_e}{P_a}, 0.76, 1.0\right), & \frac{P_e}{P_a} < 0.75 \\ 1, & \frac{P_e}{P_a} \geq 0.75. \end{cases} \quad (120)$$

The effective thrust coefficient in refined mode is therefore

$$C_{F,\text{eff}} = C_{F,\text{amb}} \eta_{\text{geom}} \eta_{\text{sep}}. \quad (121)$$

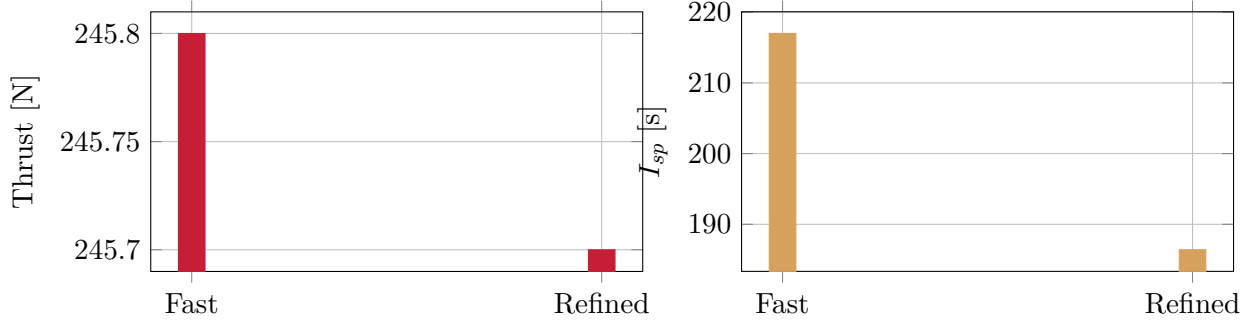


Figure 12: Fast-versus-refined flow-mode comparison for the packaged sample case.

12.2 Area Relations

The chamber and exit areas are

$$A = \pi \left(\frac{d}{2} \right)^2 \quad (122)$$

after conversion of diameter to meters. In the fast mode, StanThrust does not directly size the throat from thermodynamic choking; instead it uses the geometry-derived throat fraction

$$\phi_t = \text{clip}\left(0.21 + 0.015(\chi_f - 0.5) - 0.010(\tau_{ox} - 0.6) + 0.01\mathcal{K}_{\text{pump}} - 0.005\mathcal{K}_{\text{pressure-fed}}, 0.16, 0.24\right) \quad (123)$$

and then

$$A_t = A_c \phi_t. \quad (124)$$

In the refined mode, the throat area is taken directly from the solved throat diameter:

$$A_{t,\text{refined}} = \pi \left(\frac{d_t}{2} \right)^2. \quad (125)$$

12.3 Characteristic Velocity

The ideal characteristic velocity used by the solver is

$$c_{\text{ideal}}^* = \frac{\sqrt{RT_c}}{\gamma \sqrt{\left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{\gamma-1}}}} \quad (126)$$

and the empirical efficiency applied to it is

$$\eta_{c^*} = \text{clip}\left([0.90 + 0.04\chi_f + 0.03\eta_{\text{regen}} + 0.01\eta_{\text{pump}} - 0.01\eta_{\text{pressure-fed}} - 0.03\tau_{ox}] \eta_{\text{comb}} \eta_{\text{thermo}}, 0.72, 1.15\right) \quad (127)$$

where η_{comb} is the solver-assumption combustion efficiency and η_{thermo} is the provider-specific thermochemistry efficiency factor.

The effective characteristic velocity is then

$$c^* = \eta_{c^*} c_{\text{ideal}}^*. \quad (128)$$

12.4 Area-Mach Relation and Vacuum Thrust Coefficient

The code solves the supersonic branch of the isentropic area-Mach relation:

$$\frac{A}{A^*} = \frac{1}{M} \left[\left(\frac{2}{\gamma+1} \right) \left(1 + \frac{\gamma-1}{2} M^2 \right) \right]^{\frac{\gamma+1}{2(\gamma-1)}}. \quad (129)$$

For the exit area ratio $\varepsilon = A_e/A_t$, the solver numerically finds $M_e > 1$ by bisection.

The resulting exit pressure ratio is

$$\frac{P_e}{P_c} = \left(1 + \frac{\gamma-1}{2} M_e^2 \right)^{-\gamma/(\gamma-1)} \quad (130)$$

and the vacuum thrust coefficient before nozzle losses is

$$C_{F,\text{vac}} = \sqrt{\frac{2\gamma^2}{\gamma-1} \left(\frac{2}{\gamma+1} \right)^{\frac{\gamma+1}{\gamma-1}} \left[1 - \left(\frac{P_e}{P_c} \right)^{(\gamma-1)/\gamma} \right]} + \left(\frac{P_e}{P_c} \right) \varepsilon. \quad (131)$$

12.5 Nozzle-Loss Model

StanThrust computes a geometric nozzle-loss multiplier from the actual contour envelope. In the fast mode, the physical nozzle half-angle is

$$\alpha = \tan^{-1} \left(\frac{r_e - r_t}{L_n} \right). \quad (132)$$

The base divergence efficiency is

$$\eta_{\text{div},0} = \text{clip} \left(\frac{1 + \cos \alpha}{2}, 0.82, 1.0 \right). \quad (133)$$

If α exceeds the reference half-angle α_{ref} , the solver applies an excess-angle penalty:

$$\xi_\alpha = \frac{\alpha}{\alpha_{\text{ref}}} - 1 \quad (134)$$

$$\eta_{\text{div}} = \text{clip}(\eta_{\text{div},0}(1 - 0.08 \xi_\alpha), 0.78, 1.0). \quad (135)$$

With

$$\Lambda = \frac{L_n}{d_t} \quad (136)$$

the boundary-layer penalty becomes

$$\Delta_{\text{BL}} = k_{\text{BL}} \max(0, (\sqrt{\varepsilon} - 1) \max(0.35, 1.85 - \Lambda)) \quad (137)$$

where k_{BL} is the solver-assumption nozzle boundary-layer loss factor. The corresponding efficiency is

$$\eta_{\text{BL}} = \text{clip}(1 - \Delta_{\text{BL}}, 0.88, 0.997). \quad (138)$$

With discharge coefficient C_d and an assumption-level nozzle efficiency η_n , the fast-mode geometry and overall losses are

$$\eta_{\text{geom}} = \eta_{\text{div}} \eta_{\text{BL}} C_d \quad (139)$$

$$\eta_{\text{overall}} = \text{clip}(\eta_n \eta_{\text{geom}}, 0.68, 1.0). \quad (140)$$

The effective thrust coefficient used in the solver is therefore

$$C_{F,\text{eff}} = C_{F,\text{vac}} \eta_{\text{overall}}. \quad (141)$$

In the refined mode, StanThrust replaces the single-angle interpretation with contour-derived local wall angles:

$$\bar{\alpha} = \frac{1}{N} \sum_{i=1}^N \alpha_i, \quad \alpha_{\text{max}} = \max_i \alpha_i \quad (142)$$

where each α_i is computed from the local slope between adjacent sampled contour points on the diverging branch. A curvature-efficiency term is then introduced:

$$\eta_{\text{curv}} = \text{clip}\left(1 - \frac{1}{720} \sum_{i=2}^N |\alpha_i - \alpha_{i-1}|, 0.90, 0.998\right) \quad (143)$$

and the refined geometry efficiency becomes

$$\eta_{\text{geom,refined}} = \eta_{\text{div}} \eta_{\text{curv}} \eta_{\text{BL}} C_d. \quad (144)$$

For clarity, the implemented code multiplies these factors:

$$\eta_{\text{geom,refined}} = \eta_{\text{div}} \cdot \eta_{\text{curv}} \cdot \eta_{\text{BL}} \cdot C_d. \quad (145)$$

For the default case in this report,

$$\eta_{\text{geom}} = 0.8739, \quad \eta_{\text{overall}} = 0.8739.$$

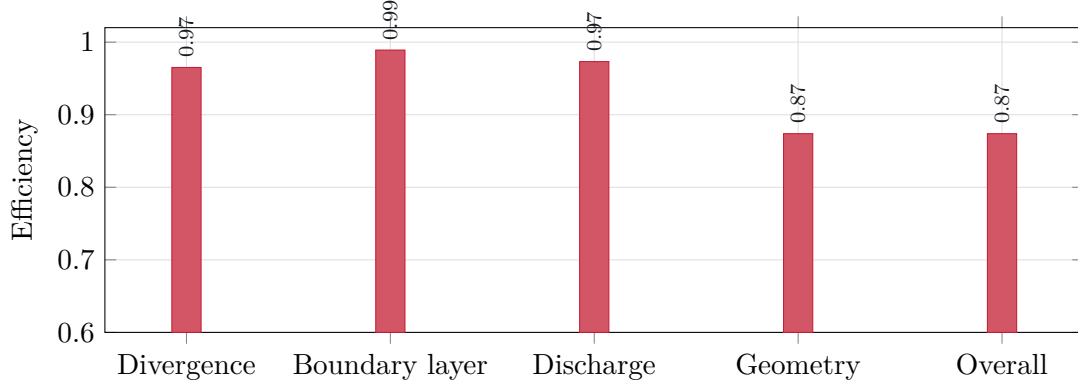


Figure 13: Nozzle-loss decomposition used to reduce the vacuum thrust coefficient to the effective thrust coefficient.

12.6 Chamber-Pressure Iteration

The solver perturbs target thrust before the iteration using

$$\phi_F = \text{clip}(0.97 + 0.02\chi_f - 0.015\tau_{ox} + 0.01\mathcal{K}_{\text{pump}} - 0.01\mathcal{K}_{\text{pressure-fed}}, 0.92, 1.04) \quad (146)$$

$$F_{\text{design}} = \phi_F F_{\text{target}}. \quad (147)$$

The chamber-pressure target implied by the nozzle model is

$$P_{c,\text{target}} = \frac{F_{\text{design}}}{C_{F,\text{eff}} A_t}. \quad (148)$$

StanThrust then iterates chamber pressure with a constant linear blend:

$$P_c^{(k+1)} = 0.72 P_c^{(k)} + 0.28 P_{c,\text{target}}. \quad (149)$$

At each iteration, chamber density is

$$\rho_c^{(k)} = \frac{P_c^{(k)}}{RT_c} \quad (150)$$

the mass flow is

$$\dot{m}_p^{(k)} = \frac{P_c^{(k)} A_t}{c^*} \quad (151)$$

and thrust is

$$F^{(k)} = C_{F,\text{eff}} P_c^{(k)} A_t. \quad (152)$$

The chamber velocity estimate is

$$u_c^{(k)} = \frac{\dot{m}_p^{(k)}}{\rho_c^{(k)} A_c} \quad (153)$$

and the relative convergence error is

$$e^{(k)} = \frac{|F^{(k)} - F_{\text{design}}|}{\max(1, F_{\text{design}})}. \quad (154)$$

The loop stops when $e^{(k)}$ falls below the configured convergence tolerance or the iteration limit is reached.

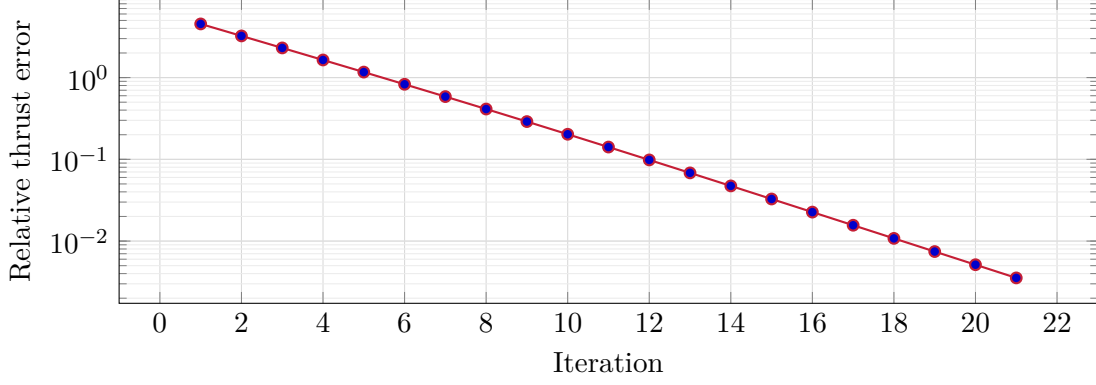


Figure 14: Default-case convergence trace of the chamber iteration. The current report case converged in 21 iterations.

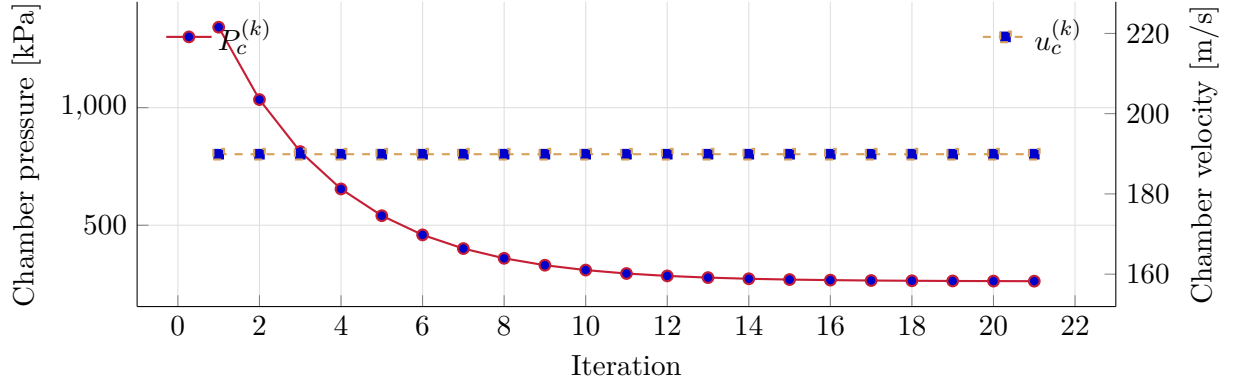


Figure 15: State evolution through the chamber-pressure iteration.

12.7 Exit State and Final Performance

After convergence, the solver computes

$$u_e = c^* C_{F,\text{vac}} \quad (155)$$

$$T_e = \frac{T_c}{1 + \frac{\gamma-1}{2} M_e^2} \quad (156)$$

$$F = C_{F,\text{eff}} P_c A_t \quad (157)$$

$$I_{sp} = \frac{F}{\dot{m}_p g_0} \quad (158)$$

$$I = F \max(0.5, t_b). \quad (159)$$

13 Axial Proxy Fields

For visualization and exports, StanThrust creates a simple axial field model across the entire engine length. If

$$\xi = \frac{x}{L_{\text{stack}}} \in [0, 1],$$

then the report pressure and temperature profiles are constructed as

$$P(x) = \max(80, P_c(1 - 0.78 \xi)) \quad (160)$$

$$T(x) = \max(220, T_c(1 - 0.46 \xi)) \quad (161)$$

$$u(x) = u_e(0.08 + 0.92 \xi). \quad (162)$$

These are display-oriented field proxies, not a discretized conservation-law solution.

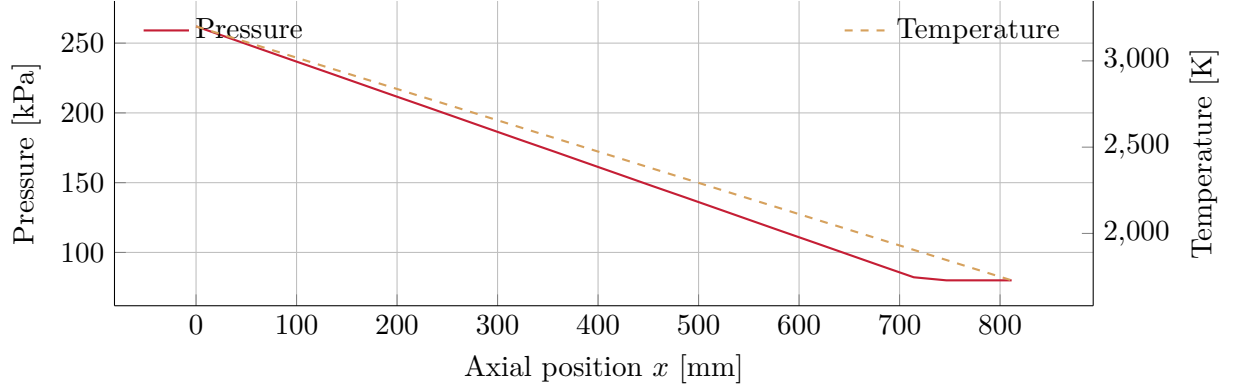


Figure 16: Representative axial fields used for visualization and export.

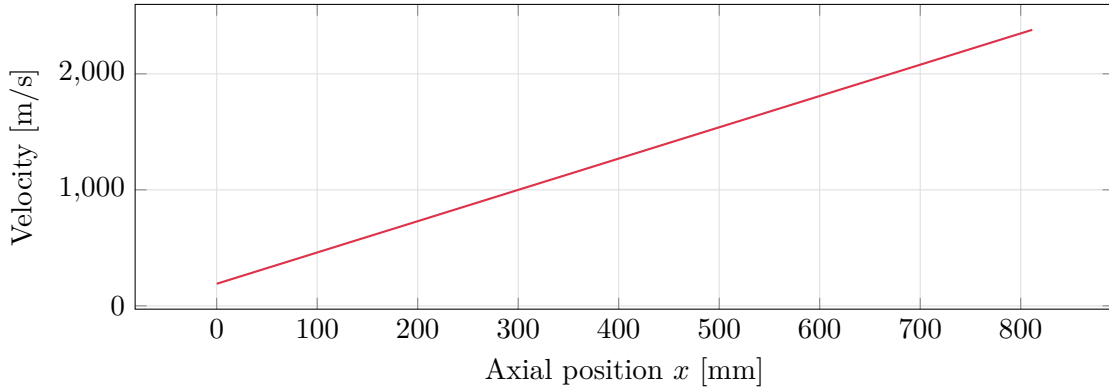


Figure 17: Axial velocity ramp used in the report's simplified field model.

14 Representative Output

For the default report case, the current solver returned approximately

$$P_c = 262.0 \text{ kPa}, \quad F = 245.7 \text{ N}, \quad I_{sp} = 186.4 \text{ s}, \quad c^* = 1388.7 \text{ m/s}, \quad M_e = 3.40.$$

The output should be read as the result of the staged closure above:

1. geometry and mass heuristics establish d_t , L_n , and flow scales;
2. pressure closure enforces architecture feasibility;
3. thermochemistry provides (γ, R, T_c) ;
4. the quasi-1D solver returns P_c , $\dot{m}_p$, F , and I_{sp} .

15 Validation Workflow

StanThrust validation is intentionally split into several checks rather than a single pass/fail number. The public benchmark reconstructions below compare the solver against published collegiate engines where enough information exists to rebuild an operating point. The internal regression baselines check repeatability of representative solved cases. The generated report figures check that the documentation data are regenerated from the same implementation that drives the desktop app. Finally, the packaging workflow runs the Python test suite before building native Windows and macOS distribution artifacts.

For a solver change to be considered healthy, the expected path is:

1. regenerate this report's CSV assets with `docs/overleaf_solver_report/generate_assets.py`;
2. run the automated test suite, including benchmark regression and render-geometry field checks;
3. inspect the live Qt diagnostics log for solver-stage status and error codes;
4. build the platform installer from the tracked packaging scripts or the GitHub packaging workflow.

This keeps method documentation, UI-visible geometry, and packaged application behavior tied to the same code path.

16 Benchmark Against Real Student Engines

The most useful benchmark is not a simple side-by-side list of thrust levels. A better engineering check is to reconstruct a small set of public student engines, run those reconstructed cases through the same StanThrust solver used for the default design, and then compare the solver output against the published operating point.

This report therefore uses a two-level benchmark:

1. a *scale comparison*, which shows where the packaged default case sits relative to public collegiate hardware;
2. a *reconstructed case study set*, where published engine data are turned into StanThrust input states and solved directly.

For unit conversion,

$$F[\text{N}] = 4.44822 F[\text{lbf}], \quad P[\text{kPa}] = 6.89476 P[\text{psi}]. \quad (163)$$

When a public page reports thrust and specific impulse but not mass flow, the missing reference quantity is reconstructed with

$$\dot{m}_{\text{ref}} = \frac{F_{\text{ref}}}{I_{sp,\text{ref}} g_0}. \quad (164)$$

When thrust and mass flow are reported but specific impulse is not, the corresponding reference specific impulse is

$$I_{sp,\text{ref}} = \frac{F_{\text{ref}}}{\dot{m}_{\text{ref}} g_0}. \quad (165)$$

For any compared quantity y , the report uses the signed percent error

$$\epsilon_y = 100 \frac{y_{\text{StanThrust}} - y_{\text{ref}}}{y_{\text{ref}}}. \quad (166)$$

The benchmark definitions themselves are treated as first-class solver inputs rather than report-only prose. StanThrust now keeps:

1. a canonical public benchmark registry, which stores the collegiate engine anchors and reconstruction assumptions in a machine-readable form; and
2. a canonical internal regression registry, which stores solver-owned baseline cases together with acceptable output envelopes.

That separation matters. The public cases test whether StanThrust lands in the right engineering neighborhood for published engines, while the internal regression cases test whether future code changes preserve the tool’s current behavior on representative in-house sizing problems.

Internal Regression Baselines

Table 3 lists the internal baseline cases that StanThrust now uses as automated regression anchors. Each row reports the current solved value together with the allowable envelope stored in the code-base. If a future solver change pushes a metric outside that envelope, the benchmark regression tests fail even if the public collegiate comparisons still compile.

Table 3: Internal StanThrust regression baseline cases and allowable envelopes.

Case	F [N]	F range [N]	P_c [kPa]	P_c range [kPa]	ϵ	ϵ range	L_{stack} [mm]	L_{stack} range [mm]
Default pressure-fed sample	210.5	190.0 to 230.0	1640.0	1450.0 to 1800.0	11.61	10.8 to 12.2	972.7	920.0 to 1025.0
Pump-fed regen midscale case	841.0	780.0 to 905.0	2159.5	1950.0 to 2350.0	11.7	10.9 to 12.4	1156.8	1090.0 to 1215.0
Film-cooled pressure-fed large case	1559.9	1450.0 to 1685.0	2140.4	1900.0 to 2350.0	10.61	9.8 to 11.4	1726.1	1625.0 to 1815.0

The internal table complements the public benchmark rather than replacing it. A solver can drift in subtle ways that do not immediately move the public reconstructions very far, especially when those public cases contain inferred assumptions. The internal baselines therefore provide a tighter local regression gate for thrust level, chamber pressure, expansion ratio, and overall packaging.

Scale Comparison

Table 4: Public student-engine reference set used for benchmark reconstruction.

Engine	Team	Propellants	Thrust [lbf]	Chamber pressure [psi]	Source
Elysium	Texas A&M RED	Ethanol / N ₂ O	300	150	[A]
Juno	ERPL	LOX / Ethanol	350	450	[B]
Iron Lotus	BURPG	IPA / N ₂ O	2553	500	[C]

Sources. [A] Texas A&M RED, Elysium project page. [B] ERPL, Juno engine page. [C] Boston University Rocket Propulsion Group, Iron Lotus page. [D] Boston University Rocket Propulsion Group, Lotus Dev 2 page.

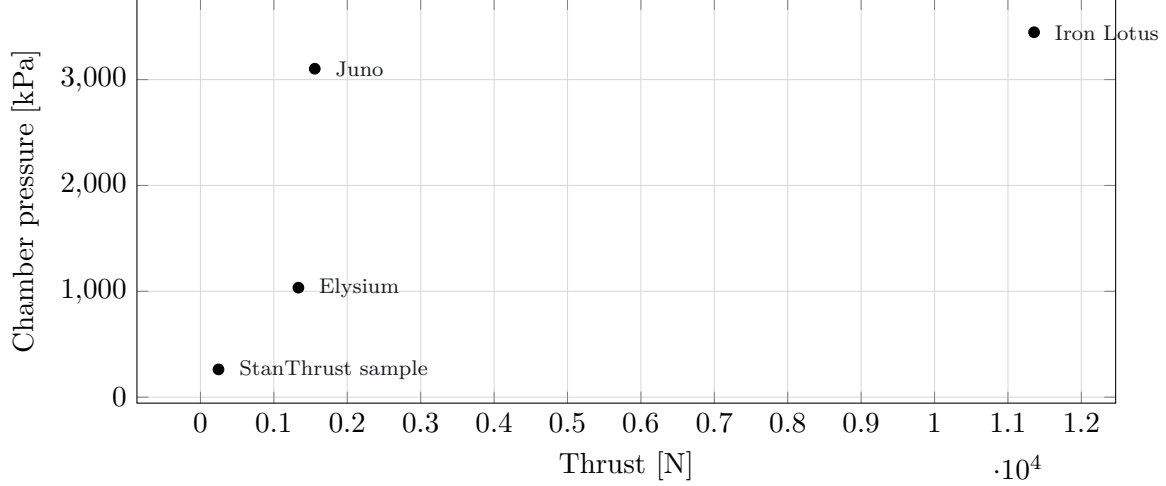


Figure 18: Scale comparison between the packaged StanThrust sample case and the public collegiate engine reference points.

The scale plot immediately shows that the packaged default case is much smaller than the collegiate engines listed in Table 4. That is useful for context, but it is not yet a solver validation. A more meaningful check is to reconstruct each public engine as a StanThrust case and compare the solved operating point directly.

Reconstructed Case Setup

Table 5 lists the public quantities used as benchmark anchors. Table 6 lists the additional modeling assumptions required to build complete StanThrust inputs.

Table 5: Published operating-point anchors used to reconstruct each benchmark engine. Blank fields indicate quantities not reported directly on the public reference page and therefore inferred with Eqs. (164) through (165) when needed.

Engine	Fuel	Oxidizer	F_{ref} [N]	$P_{c,\text{ref}}$ [kPa]	$t_{b,\text{ref}}$ [s]	$\dot{m}_{\text{ref}}$ [kg/s]	$I_{sp,\text{ref}}$ [s]
Elysium	ethanol	nitrous oxide	1334.466	1034.214	3.5		
Juno	ethanol	liquid oxygen	1556.878	3102.641	3.0	0.66	240.54141
Iron Lotus	isopropyl alcohol	nitrous oxide	11356.31	3447.379	3.0	5.48825	211.0

Table 6: StanThrust modeling assumptions used to close the reconstructed benchmark cases.

Engine	Modeled O/F	Feed architecture	Regen	Film
Elysium	4.0	pressure-fed	no	no
Juno	1.3	pressure-fed	no	no
Iron Lotus	4.0	pressure-fed	no	yes

Case notes.

- **Elysium.** Public thrust, chamber pressure, and burn duration are used directly from [A]. The feed architecture and O/F are not stated on the public page, so the reconstruction uses a pressure-fed ethanol/ N_2O state with $O/F = 4.0$.

- **Juno.** Public thrust, chamber pressure, mass flow, and O/F are taken directly from [B]. The benchmark state assumes a short 3 s firing window and a pressure-fed architecture because the public page does not list those inputs explicitly.
- **Iron Lotus.** Public thrust, chamber pressure, burn duration, and specific impulse are taken from [C]. The pressure-fed assumption and $O/F = 4.0$ are guided by the closely related Lotus Dev 2 program [D]. Film cooling is enabled because the public Iron Lotus description explicitly mentions supplemental film cooling.

Individual StanThrust Runs

Each public engine is run as its own StanThrust case rather than being reduced to a single scale point. That matters because a static benchmark table can compare thrust and chamber pressure, but it cannot close the missing mass-flow, specific-impulse, nozzle, chamber, and packaging quantities. The individual runs in Table 7 report both comparison quality and the amount of solved engineering state produced by the same solver call.

Table 7: Individual reconstructed benchmark runs. The comparable-stat count includes thrust, chamber pressure, mass flow, and specific impulse when a public reference value is available. The solved-field count covers thrust, chamber pressure, mass flow, specific impulse, throat diameter, exit diameter, expansion ratio, chamber length, and total stack length.

Engine	Compared stats	Mean abs. error [%]	Max abs. error [%]	Solved ε	L_{stack} [mm]	Solved fields
Elysium	2	13.91	24.32	10.89	904.1	9
Juno	4	25.56	58.99	11.8	973.5	9
Iron Lotus	4	8.47	26.6	9.46	2113.7	9

This is where the reconstructed StanThrust cases are more useful than a flat reference list. The solver does not simply restate the published thrust. It builds a full engine state, exposes which values are directly comparable, and leaves a reproducible trail of derived geometry that can be checked again after future code changes.

Reconstructed Operating-Point Comparison

Figure 19 compares the published and reconstructed StanThrust operating points. The purpose of this plot is not to claim exact duplication of the student engines; instead, it shows whether the reduced-order model lands in the correct performance neighborhood once the public hardware is translated into the StanThrust input vocabulary.

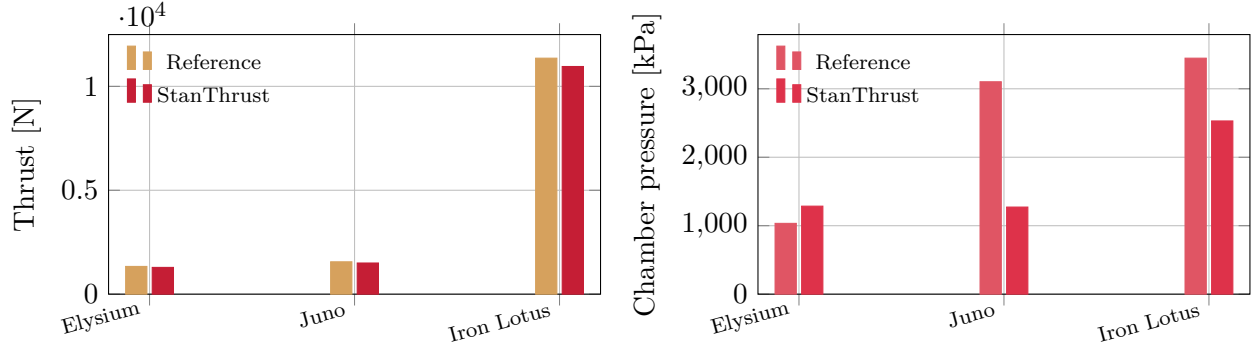


Figure 19: Published operating-point anchors versus StanThrust reconstructed operating points.

Figure 20 converts the same comparison into signed percent errors using Eq. (166). This is the most compact way to judge how the reduced-order solver is behaving across engines with very different scales.

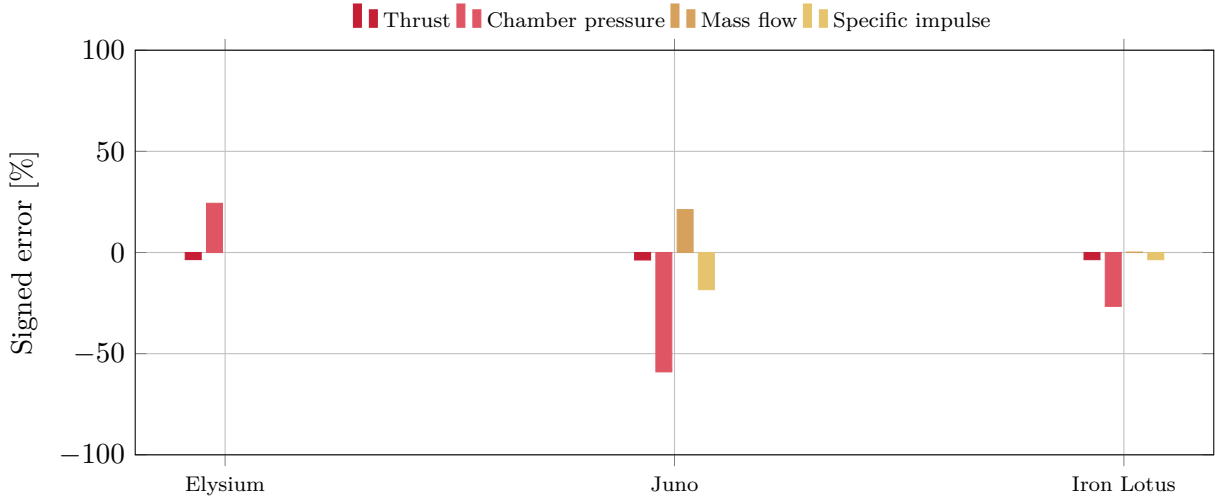


Figure 20: Signed percent errors for the reconstructed benchmark cases. Empty bars correspond to quantities not published directly and therefore not compared on the original source page.

Generated Geometry for the Reconstructed Cases

The operating-point comparison above checks whether StanThrust reproduces the right scale of thrust, chamber pressure, mass flow, and specific impulse. The next question is what geometry the solver generates in order to support those states. Because the public collegiate engine pages do not generally publish full chamber and nozzle dimensions, the geometry comparison is necessarily one-sided: the report can show the geometry implied by the solver, but it cannot always compute a direct geometric error against the real hardware.

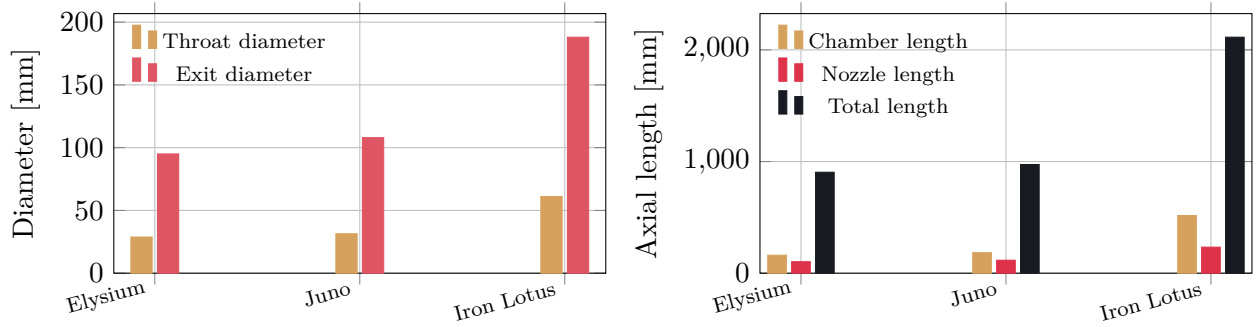


Figure 21: Geometry generated by StanThrust for the reconstructed benchmark cases.

The generated geometry plots are useful in two ways. First, they show whether the solver is producing dimensions that are plausible for the class of engine being reconstructed. Second, they reveal which geometry variables StanThrust changes most aggressively when it is asked to match higher thrust and chamber pressure. In practice, the exit diameter, chamber length, and total axial length tend to separate the small and large engines most clearly.

Taken together, the benchmark reconstruction should be read as a *behavioral validation* rather than a strict identification exercise. If the published operating-point data are sparse, the modeled architecture, O/F , and cooling choices inevitably carry some uncertainty. Even so, the reconstructed cases are much more informative than a simple benchmark table because they show how the solver responds when asked to emulate real student-engine operating points.

17 Scope and Limits

Despite the large number of equations above, StanThrust remains a reduced-order tool. The equations are fast algebraic closures, not first-principles time-marching simulations. In particular:

- the feed solver now time-marches pressure and margin over the burn, but it is still a reduced-order lumped model rather than a valve-by-valve fluid network or full pressurant thermodynamics model;
- chamber and nozzle performance are quasi-one-dimensional with empirical loss factors;
- thermal quantities are bounded engineering proxies;
- pump, injector, and cooling dimensions are sizing heuristics rather than detailed component designs;
- the axial field plots are visualization-oriented profiles, not CFD field solutions.

Within those limits, the implementation is internally consistent and well suited to rapid trade studies, packaging iteration, and architecture comparison.